Université Paris Cité

Programmation Java
TP n° 4 - Quizz

Le but de sujet est de faire un programme informatique permettant de lancerJeu des questionnaires notés.
L'execution du programme donnerait par exemple quelque chose comme :

```
--------------------------------------------------------------------------------
    Quel est le nom du prof de Prog2 ?  (pour 5 points)
--------------------------------------------------------------------------------
Charbit
Bonne réponse
--------------------------------------------------------------------------------
    Citez une aglomération de plus de 25 millions d'habitants  (pour 10 points)
--------------------------------------------------------------------------------
Paris
Mauvaise réponse
--------------------------------------------------------------------------------
    Quel age a le prof de Prog2? (a 5 près) (pour 5 points)
--------------------------------------------------------------------------------
20
Mauvaise réponse

--------------------------------------------------------------------------------
    Lequel de ces films n'est pas un film d'Alfred Hitchcock (pour 2 points)
      1) Les Oiseaux
      2) Les Affranchis
      3) Frenzy
      4) Sueurs Froides
--------------------------------------------------------------------------------
3
Mauvaise réponse
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
FINI. Vous avez marque 5 points sur 22 possibles
```

On voit dans cet exemple que les questions posées peuvent être de types différents. Les deux premières attendent une réponse textuelle (notez que la première a une bonne réponse, la seconde en a plusieurs possibles), la troisième demande une réponse numérique, avec une marge d'erreur autorisée, et la dernière est une question à choix multiple.
C'est pourquoi on va écrire une classe mère abstraite `Question` qui va décrire ce qui est en commun pour tous les types, puis ensuite plusieurs sous classes différentes représentant les types de questions possibles afin que la fonction qui vérifie si une réponse est bonne ait un comportement adapté.
Pour la classe `Question` on utilisera la classe abstraite suivante :

```java
public abstract class Question{

  // doit retourner une chaine de caractères contenant le texte de la question
  public abstract String getIntitule();

  //doit retourner true si la chaine s passée en argument est une bonne réponse pour
    cette question
  public abstract boolean isBonneReponse(String s);
}
```

On rappelle qu'on vous recommande d'écrire un fichier .java pour chaque nouvelle classe

Exercice 1 : Le Quizz

Récupérez la classe `Question` puis écrivez une classe `Prog` qui contient une méthode statique

```
public static void lancerJeu(ArrayList<Question> liste)
```

qui fait se dérouler le jeu basé la liste de `Question` passée en argument, c'est à dire qui parcourt la liste et pour chaque question :

- pose la question
- récupère la réponse
- vérifie la réponse et on dit si c'est correct ou pas

Ensuite à la fin on dit à l'utilisateur combien de bonnes réponses il a eu.
Bien sûr vous allez utiliser les fonctions `getIntitule` et `isBonneReponse` du type abstrait `Question`.

Exercice 2 : Questions à réponse textuelles

Dans cet exercice, on va définir une sous classe `QuestionTexte` de la classe `Question`, dont la réponse est du type textuel (comme la première de l'exemple introductif). Elle possèdera probablement deux champs de type `String` pour stocker l'intitule de la question et la bonne réponse attendue.

1. Écrire cette classe avec un constructeur adapté et en redéfinissant les méthodes `String getIntitule()` (pour qu'elle retourne un intitule) et `public boolean isBonneReponse(String r)` pour qu'elle renvoie `true` si et seulement si le `String r` est la bonne réponse. Il vous faudra aussi un constructeur permettant d'initialiser les champs `intitule` et `reponseCorrecte`

2. Maintenant que l'on dispose d'une classe concrète qui étend `Question`, ajoutez dans `Prog` un `main` dans lequel vous créez une liste de questions textuelles pour ensuite tester `lancerJeu` sur cette liste.

Exercice 3 : Questions à réponse numériques

On veut définir un nouveau type de question `QuestionNum`, des questions à réponse numérique entière. Pour ces questions, on autorise une marge d'erreur possible (comme la deuxième dans l'exemple du début).

1. Ecrire cette classe, en trouvant quels champs ajouter dans cette classe pour qu'elle puisse fonctionner correctement.

2. Redéfinir la méthode `String getIntitule()` pour qu'elle précise la marge d'erreur autorisée (cf. exemple)

3. Redéfinir dans cette classe la méthode `public boolean isBonneReponse(String r)` pour qu'elle renvoie `true` si et seulement le `String` correspond à un entier qui est dans les bornes autorisées. Il faut utiliser la fonction `int Integer.parseInt(String r)` qui construit un `int` à partir d'une chaine de caractères (si la chaine ne correspond pas à un entier, elle retourne une `NumberFormatException`).

Exercice 4 : Des Fonctionnalités supplémentaires

1. Proposer une façon d'ajouter la gestion des points pour chaque question

2. Dans l'exemple introductif, on voit que la seconde question attend une réponse de type textuelle mais plusieurs bonnes réponses sont possibles. Comment modifier la classe `QuestionTexte` pour permettre ce genre de questions, et sans avoir a trop modifié du code déjà écrit (notamment les appels au constructeur de type `new QuestionTexte(...)` ?

3. Comment faire des questions de type une réponse numérique réelle (avec marge d'erreur en pourcentage) ? Exemple : "quel est la valeur de Pi (à 5% près) ?"

4. Proposez une façon d'avoir des questions à choix multiple comme dans l'exemple introductif

Exercice 5 : Version Graphique

En Java, il existe des fonctions statiques assez simples d'utilisation qui permettent de générer des boites de dialogue avec saisie possible. Elles se situent dans la librairie `javax.swing` au sein de la classe `JOptionPane` (on écrira donc `import javax.swing.JOptionPane` en début de fichier pour les utiliser
Par exemple la méthode `int JOptionPane.showMessageDialog` permet d'ouvrir une boite de dialogue à boutons. L'appel de cette méthode est bloquant, c'est à dire que le programme s'arrête tant qu'on n'a pas appuyé sur un des boutons et on peut récupérer si nécessaire en sortie un entier qui indique sur quel bouton (typiquement OK ou CANCEL) on a appuyé.
Essayez

```
int x = JOptionPane.showMessageDialog(null, "Ceci est le message") // d'autres constructeurs
    + sophistiqués existent pour les curieux
```

Une autre très utile est `showInputDialog` qui ouvre une boite dans laquelle on peut faire une saisie et dont on peut récupérer la réposne sour forme de `String`. Essayez

```
String reponse = JOptionPane.showInputDialog(null, "Voilà ma question");
```

1. A l'aide de ces deux méthodes, ajoutez dans `Prog` une méthode `lancerJeuGraphique` qui fait se dérouler le jeu via des boites de dialogues au lieu du terminal pour les affichages et les saisies via `Scanner`. A priori il y a peu a modifier par rapport à la méthode `lancerJeu`.

Exercice 6 : Fabriquer des Quizz automatiquement à partir de fichiers texte

Dans cette partie on va essayer de générer aléatoirement un Quizz à partir d'un fichier texte qui contient plein de questions. On va écrire une classe `QuizzFactory` qui possèdera une méthode statique

```
public static ArrayList<Question> genereRandomQuizz(String nomFichier, int n)
```

qui permettra ensuite dans un `main` de générer un quizz aléatoire de la façon suivante :

```
ArrayList<Question> q = QuizzFactory.genereRandomQuizz("monfichier.txt",10); // 10 questions
    piochées au hasard dans le fichier monfichier.txt
```

Bien sûr il faut que ce fichier texte soit écrit de façon précise et codifiée pour que notre programme puisse extraire les questions.
Voici une possibilité de formatage des lignes que je vous propose : elle consiste découper la ligne avec le symbole "/" et a mettre au tout début une lettre pour indiquer le type de question :

- pour les QuestionTexte : "T/titre/valeur/bonneRep1/bonneRep2..."
- pour les QuestionNum : "N/titre/valeur/bonneRep/marge "
- pour les QuestionChoixMultiple : "M/titre/valeur/bonneRep/Choix1/Choix2/Choix3/"

Par exemple une ligne pourrait être

```
T/Donnez le nom de famille d'un membre des Beatles/1/Lennon/McCartney/Starr/Harrison
```

Regardez mon exemple de fichier "database.txt".

Récupérez le fichier `QuizzFactory.java` sur Moodle : il contient deux méthodes à compléter.

1. Complétez la méthode `Question extraireQuestion(String ligne)`. Cette méthode suppose que `ligne` est une ligne au format ci dessus et doit construire et retourner un objet de type `Question` correspondant. Pour cela vous pourrez utiliser la fonction `split` de la classe `String`. Elle prend en entrée un `String` correspondant a un délimiteur et retourne un tableau de `String`.

```
String[] tabligne = ligne.split("/");
```

On obtient un tableau dont chaque entrée correspond à un découpage de la ligne autour du caractère de séparation `"/"` passé en argument de `split`.

2. Complétez maintenant la méthode `generateRandomQuizz` qui prend en argument un nom de fichier et un entier `n` et qui retourne une `ArrayList<Question>` avec `n` questions piochées au hasar dans le fichier. Elle contient déjà la ligne suivante :

```
List<String> listelignes = Files.readAllLines(Paths.get(nomFichier));
```

Si `nomFichier` est un nom de fichier texte (par exemple "questions.txt") présent dans votre répertoire, alors l'instruction ci dessus va découper le fichier ligne par ligne et construire une liste `listelignes` de `String` correspondant à chaque ligne (`listelignes.get(0)` est la première ligne, `listeslignes.get(1)` la suivante, etc...). Pour cette ligne vous devrez ajouter les imports suivants au début du fichier.

```
import java.nio.file.Files;
import java.nio.file.Paths;
```

Il faut ajouter du code pour que `n` fois de suite, on pioche au hasard une ligne dans la liste, on lui applique la fonction de la qeustion précédente pour créer une `Question` puis on l'ajoute à la liste qu'on retourne à la fin de la fonction

3. Récupérez mon fichier `database.txt` (vous pouvez rajouter des questions) et écrivez maintenant votre programme final. Il s'agira d'un main qui piochera dans le fichier `database` pour générer un quizz de 10 questions, puis fera se dérouler le quizz.